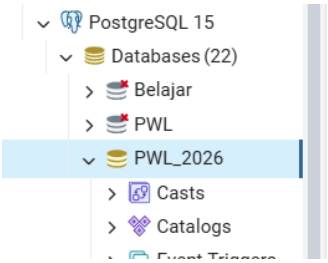
KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
https://jti.polinema.ac.id

Mata Kuliah	:	Pemrograman Web Lanjut
Program Studi	:	D4 – Teknik Informatika
Semester	:	4

Kelas	:	2D
NIM	:	244107020150
Nama	:	Gaduh Prakoso
Jobsheet Ke-	:	3

Laporan Jobsheet

Praktikum Ke-1

Langkah	Jawaban/Deskripsi
1	Buka aplikasi phpMyAdmin, dan buat database baru dengan nama PWL_POS 
2	Buka aplikasi VSCode dan buka folder project PWL_POS yang sudah kita buat
3	Copy file .env.example menjadi .env
4	Buka file .env, dan pastikan konfigurasi APP_KEY bernilai. Jika belum bernilai silahkan kalian generate menggunakan php artisan.

5	Edit file .env dan sesuaikan dengan database yang telah dibuat
	`DB_CONNECTION=pgsql`
	`DB_HOST=127.0.0.1`
	`DB_PORT=5432`
	`DB_DATABASE=PWL_2026`
	`DB_USERNAME=postgres`
	`DB_PASSWORD=12345678`

Praktikum Ke-2

Langkah	Jawaban/Deskripsi
	2.1 Pembuatan file migrasi tanpa relasi
1	Buka terminal VSCode kalian, untuk yang di kotak merah adalah default dari laravel

2	Kita buat file migrasi untuk table m_level dengan perintah
	`>php artisan make:migration create_m_level_table --create=m_level`
	`<?php`
	`use Illuminate\Database\Migrations\Migration;`
	`use Illuminate\Database\Schema\Blueprint;`
	`use Illuminate\Support\Facades\Schema;`
	`return new class extends Migration`
	`{`
	`/**`
	` * Run the migrations.`
	` */`
	`public function up(): void`
	`{`
	`Schema::create('m_level', function (Blueprint $table): void {`
	`$table->id();`
	`$table->timestamps();`
	`});`
	`}`
	`/**`
	` * Reverse the migrations.`
	` */`
	`public function down(): void`
	`{`
	`Schema::dropIfExists('m_level');`
	`}`
	`};`

3	Kita perhatikan bagian yang di kotak merah, bagian tersebut yang akan kita modifikasi sesuai desain database yang sudah ada
4	Simpan kode pada tahapan 4 tersebut, kemudian jalankan perintah ini pada terminal VSCode untuk melakukan migrasi `>php artisan migrate`
5	Kemudian kita cek di phpMyAdmin apakah table sudah ter-generate atau belum
6	Buat table database dengan migration untuk table m_kategori yang sama-sama tidak memiliki foreign key
7	Laporkan hasil Praktikum-2.1 ini dan commit perubahan pada git.
	2.2 Pembuatan file migrasi dengan relasi
1	Buka terminal VSCode kalian, dan buat file migrasi untuk table m_user

2	Buka file migrasi untuk table m_user, dan modifikasi seperti berikut `>php artisan make:migration create_m_user_table --table=m_user`

3	Simpan kode program Langkah 2, dan jalankan perintah php artisan migrate. Amati apa yang terjadi pada database.
	`<?php`
	`use Illuminate\Database\Migrations\Migration;`
	`use Illuminate\Database\Schema\Blueprint;`
	`use Illuminate\Support\Facades\Schema;`
	`return new class extends Migration`
	`{`
	`/**`
	`* Run the migrations.`
	`*/`
	`public function up(): void`
	`{`
	`Schema::create(table: 'm_user', callback: function (Blueprint $table): void {`
	`$table->id(column: 'user_id');`
	`$table->unsignedBigInteger(column: 'level_id')->index(); // indexing untuk ForeignKey`
	`$table->string(column: 'username', length: 20)->unique(); // unique untuk memastikan tidak ada username yang sama`
	`$table->string(column: 'nama', length: 100);`
	`$table->string(column: 'password');`
	`$table->timestamps();`
	`// Mendefinisikan Foreign Key pada kolom level_id mengacu pada kolom level_id di tabel m_level`
	`$table->foreign(columns: 'level_id')->references(columns: 'level_id')->on(table: 'm_level');`
	`});`
	`}`
	`/**`
	`* Reverse the migrations.`
	`*/`
	`public function down(): void`
	`{`
	`Schema::dropIfExists(table: 'm_user');`
	`}`
	`};`

4	Buat table database dengan migration untuk table-tabel yang memiliki foreign key
5	Jika semua file migrasi sudah di buat dan dijalankan maka bisa kita lihat tampilan designer pada phpMyAdmin seperti berikut
6	Laporkan hasil Praktikum-2.2 ini dan commit perubahan pada git.

Praktikum Ke-3

Langkah	Jawaban/Deskripsi
1	Kita akan membuat file seeder untuk table m_level dengan mengetikkan perintah `>php artisan make:seeder leverlseeder`
2	Selanjutnya, untuk memasukkan data awal, kita modifikasi file tersebut di dalam function run()

	``php
	namespace Database\Seeders;
	
	use Illuminate\Database\Console\Seeds\WithoutModelEvents;
	use Illuminate\Database\Seeder;
	use Illuminate\Support\Facades\DB; // Pastikan ini ada
	
	0 references | 0 implementations
	class LevelSeeder extends Seeder
	{
	/**
	* Run the database seeds.
	*/
	0 references | 0 overrides
	public function run(): void
	{
	$data = [
	['level_id' => 1, 'level_kode' => 'ADM', 'level_nama' => 'Administrator'],
	['level_id' => 2, 'level_kode' => 'MNG', 'level_nama' => 'Manager'],
	['level_id' => 3, 'level_kode' => 'STF', 'level_nama' => 'Staff/Kasir'],
	]; // Data sesuai instruksi praktikum
	
	DB::table(table: 'm_level')->insert(values: $data);
	}
	}
	``

3	Selanjutnya, kita jalankan file seeder untuk table m_level pada terminal `php artisan db:seed --class=LevelSeeder`
4	Ketika seeder berhasil dijalankan maka akan tampil data pada table m_level
5	Sekarang kita buat file seeder untuk table m_user yang me-refer ke table m_level `>php artisan make:seeder UserSeeder`
6	Modifikasi file class UserSeeder seperti berikut

```php
0 references | 0 implementations
class UserSeeder extends Seeder
{
    0 references | 0 overrides
    public function run(): void
    {
        $data = [
            [
                'user_id' => 1,
                'level_id' => 1,
                'username' => 'admin',
                'nama' => 'Administrator',
                'password' => Hash::make(value: '12345'),
            ],
            [
                'user_id' => 2,
                'level_id' => 2,
                'username' => 'manager',
                'nama' => 'Manager',
                'password' => Hash::make(value: '12345'),
            ],
            [
                'user_id' => 3,
                'level_id' => 3,
                'username' => 'staff',
                'nama' => 'Staff/Kasir',
                'password' => Hash::make(value: '12345'),
            ],
        ];
        DB::table(table: 'm_user')->insert(values: $data);
    }
}
```

7 Jalankan perintah untuk mengeksekusi class UserSeeder

```
>php artisan db:seed --class=UserSeeder
```

8 Sekarang coba kalian masukkan data seeder untuk table yang lain, dengan ketentuan seperti berikut

```php
    public function run(): void
{
    $data = [
        ['kategori_id' => 1, 'kategori_kode' => 'SNC', 'kategori_nama' => 'Snack/Makanan Ringan'],
        ['kategori_id' => 2, 'kategori_kode' => 'MNM', 'kategori_nama' => 'Minuman'],
        ['kategori_id' => 3, 'kategori_kode' => 'KSM', 'kategori_nama' => 'Kosmetik'],
        ['kategori_id' => 4, 'kategori_kode' => 'PRT', 'kategori_nama' => 'Peralatan Rumah Tangga'],
        ['kategori_id' => 5, 'kategori_kode' => 'KSH', 'kategori_nama' => 'Kesehatan'],
    ];
    DB::table('m_kategori')->insert($data);
}
}
```

	0 references | 0 overrides
	`public function run(): void`
	
	`$data = [];`
	`for ($i = 1; $i <= 10; $i++) {`
	`  $data[] = [`
	`  'penjualan_id' => $i,`
	`  'user_id' => 3, // Staff/Kasir yang melayani`
	`  'pembeli' => 'Pelanggan ' . $i,`
	`  'penjualan_kode' => 'PJN' . str_pad(string: $i, length: 3, pad_string: '0', pad_type: STR_PAD_LEFT),`
	`  'penjualan_tanggal' => now(),`
	`  ];`
	`}`
	`DB::table('t_penjualan')->insert($data);`
	
	`$data = [];`
	`for ($i = 1; $i <= 10; $i++) {`
	`  $data[] = [`
	`  'stok_id' => $i,`
	`  'barang_id' => $i,`
	`  'user_id' => 1, // Admin yang input`
	`  'stok_tanggal' => now(),`
	`  'stok_jumlah' => 100,`
	`  ];`
	`}`
	`DB::table('t_stok')->insert($data);`

9	Jika sudah, laporkan hasil Praktikum-3 ini dan commit perubahan pada git

Praktikum 4

Langkah	Jawaban/Deskripsi
1	Kita buat controller dahulu untuk mengelola data pada table m_level `>php artisan make:controller LevelController`
2	Kita modifikasi dulu untuk routing-nya, ada di PWL_POS/routes/web.php

3	Selanjutnya, kita modifikasi file LevelController untuk menambahkan 1 data ke table m_level
	
	`<?php`
	
	`namespace App\Http\Controllers;`
	
	`use Illuminate\Http\Request;`
	`use Illuminate\Support\Facades\DB; // Pastikan memanggil DB Facade`
	
	`0 references | 0 implementations`
	`class LevelController extends Controller`
	`{`
	`  0 references | 0 overrides`
	`  public function index(): string`
	`  {`
	`  // Menambahkan 1 data baru ke tabel m_level menggunakan query insert`
	`  DB::insert(query: 'insert into m_level(level_kode, level_nama, created_at) values(?, ?, ?)', bindings: ['CUS'`
	
	`  return 'Insert data baru berhasil';`
	`  }`
	`}`

4	Kita coba jalankan di browser dengan url localhost/PWL_POS/public/level dan amati apa yang terjadi pada table m_level di database, screenshot perubahan yang ada pada table m_level

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create(table: 'm_user', callback: function (Blueprint $table): void {
            $table->id(column: 'user_id');
            $table->unsignedBigInteger(column: 'level_id')->index(); // indexing untuk ForeignKey
            $table->string(column: 'username', length: 20)->unique(); // unique untuk memastikan tidak ada u
            $table->string(column: 'nama', length: 100);
            $table->string(column: 'password');
            $table->timestamps();

            // Mendefinisikan Foreign Key pada kolom level_id mengacu pada kolom level_id di tabel m_level
            $table->foreign(columns: 'level_id')->references(columns: 'level_id')->on(table: 'm_level');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists(table: 'm_user');
    }
```

5	Selanjutnya, kita modifikasi lagi file LevelController untuk meng-update data di table m_level seperti berikut
6	Kita coba jalankan di browser dengan url localhost/PWL_POS/public/level lagi dan amati apa yang terjadi pada table m_level di database, screenshot perubahan yang ada pada table m_level
7	Kita coba modifikasi lagi file LevelController untuk melakukan proses hapus data
8	Method terakhir yang kita coba adalah untuk menampilkan data yang ada di table m_level. Kita modifikasi file LevelController seperti berikut
9	Coba kita perhatikan kode yang diberi tanda kotak merah, berhubung kode tersebut memanggil view('level'), maka kita buat file view pada VSCode di PWL_POS/resources/view/level.blade.php

10	Laporkan hasil Praktikum-4 ini dan commit perubahan pada git.

Praktikum 5

Langkah	Jawaban/Deskripsi
1	Kita buat controller dahuku untuk mengelola data pada table m_kategori `php artisan make:controller KategoriController`

2	Kita modifikasi dulu untuk routing-nya, ada di PWL_POS/routes/web.php
	`<?php`
	`use App\Http\Controllers\KategoriController;`
	`use App\Http\Controllers\LevelController;`
	`use Illuminate\Support\Facades\Route;`
	`Route::get(uri: '/', action: function (): View {`
	`  return view(view: 'welcome');`
	`});`
	`Route::get(uri: '/level', action: [LevelController::class, 'index']);`
	`Route::get(uri: '/kategori', action: [KategoriController::class, 'index']);`

3	Selanjutnya, kita modifikasi file KategoriController untuk menambahkan 1 data ke table m_kategori
	`<?php`
	`namespace App\Http\Controllers;`
	`use Illuminate\Http\Request;`
	`use Illuminate\Support\Facades\DB;`
	`2 references | 0 implementations`
	`class KategoriController extends Controller`
	`{`
	`  1 reference | 0 overrides`
	`  public function index(): string`
	`  {`
	`  /* $data = [`
	`  'kategori_kode' => 'SNK',`
	`  'kategori_nama' => 'Snack/Makanan Ringan',`
	`  'created_at' => now()`
	`  ];`
	`  DB::table('m_kategori')->insert($data);`
	`  return 'Insert data baru berhasil'; */`
	`  $row = DB::table(table: 'm_kategori')->where(column: 'kategori_kode', operator: 'SNK')->update(values: ['kategori_r`
	`  return 'Update data berhasil. Jumlah data yang diupdate: ' . $row . ' baris';`
	`  }`

4	Kita coba jalankan di browser dengan url localhost/PWL_POS/public/kategori dan amati apa yang terjadi pada table m_kategori di database, screenshot perubahan yang ada pada table m_kategori

5	Selanjutnya, kita modifikasi lagi file KategoriController untuk meng-update data di table m_kategori seperti berikut

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

2 references | 0 implementations
class KategoriController extends Controller
{
    1 reference | 0 overrides
    public function index(): string
    {
        /* $data = [
            'kategori_kode' => 'SNK',
            'kategori_nama' => 'Snack/Makanan Ringan',
            'created_at' => now()
        ];
        DB::table('m_kategori')->insert($data);
        return 'Insert data baru berhasil'; */

        $row = DB::table(table: 'm_kategori')->where(column: 'kategori_kode', operator: 'SNK')->update(values: ['kategori_r
        return 'Update data berhasil. Jumlah data yang diupdate: ' . $row . ' baris';
    }
```

6	Kita coba jalankan di browser dengan url localhost/PWL_POS/public/kategori lagi dan amati apa yang terjadi pada table m_kategori di database, screenshot perubahan yang ada pada table m_kategori

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

2 references | 0 implementations
class KategoriController extends Controller
{
1 reference | 0 overrides
public function index(): string
{
    /* $data = [
        'kategori_kode' => 'SNK',
        'kategori_nama' => 'Snack/Makanan Ringan',
        'created_at' => now()
    ];
    DB::table('m_kategori')->insert($data);
    return 'Insert data baru berhasil'; */

    /* $row = DB::table('m_kategori')->where('kategori_kode', 'SNK')->update(['kategori_nama' => 'Camilan']);
    return 'Update data berhasil. Jumlah data yang diupdate: ' . $row . ' baris'; */

    $row = DB::table(table: 'm_kategori')->where(column: 'kategori_kode', operator: 'SNK')->delete();
    return 'Delete data berhasil. Jumlah data yang dihapus: ' . $row . ' baris';
}
}
```

7	Kita coba modifikasi lagi file KategoriController untuk melakukan proses hapus data

8	Method terakhir yang kita coba adalah untuk menampilkan data yang ada di table m_kategori. Kita modifikasi file KategoriController seperti berikut

```php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

2 references | 0 implementations
class KategoriController extends Controller
{
1 reference | 0 overrides
public function index(): View
{
    // ... (kode insert, update, delete sebelumnya di-comment)

    $data = DB::table(table: 'm_kategori')->get();
    return view(view: 'kategori', data: ['data' => $data]);
}
}
```

9	Coba kita perhatikan kode yang diberi tanda kotak merah, berhubung kode tersebut memanggil view(‘kategori’), maka kita buat file view pada VSCode di PWL_POS/resources/view/kategori.blade.php

```html
<!DOCTYPE html>
<html>
    <head>
        <title>Data Kategori Barang</title>
    </head>
    <body>
        <h1>Data Kategori Barang</h1>
        <table border="1" cellpadding="2" cellspacing="0">
            <tr>
                <th>ID</th>
                <th>Kode Kategori</th>
                <th>Nama Kategori</th>
            </tr>
            @foreach ($data as $d)
            <tr>
                <td>{{ $d->kategori_id }}</td>
                <td>{{ $d->kategori_kode }}</td>
                <td>{{ $d->kategori_nama }}</td>
            </tr>
            @endforeach
        </table>
    </body>
</html>
```

10	Laporkan hasil Praktikum-5 ini dan commit perubahan pada git